

Vetores e Matrizes

INTRODUÇÃO

- Variável

- Analogia: uma caixa, na qual você pode dar o nome que lhe achar conveniente, e guardar o conteúdo que desejar



- Possui um tipo (caractere, lógico, inteiro ou real)
- O valor dentro da “caixa” que pode ser alterado de acordo com a execução do algoritmo

INTRODUÇÃO'

RT

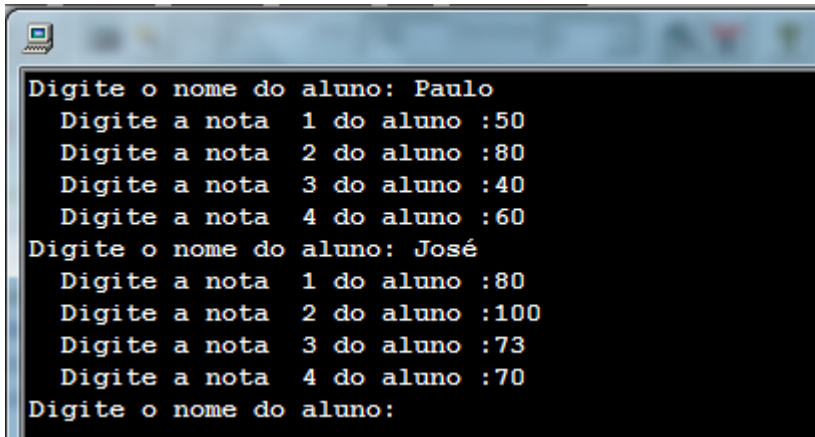
Agora imagine como ficaria na declaração de variáveis, declarando uma a uma, as 50 variáveis para o nome, depois as variáveis para as médias de cada aluno...

var

```
nota1, nota2, nota3, nota4: real  
nome_aluno1, nome_aluno2, nome_aluno3, ....., nome_aluno50: caractere  
media_aluno1, media_aluno2, media_aluno3, ....., media_aluno50: real
```

INTRODUÇÃO

- O problema começa quando se precisa declarar várias variáveis para atender a um fim.
- **PROBLEMA:** Receber o nome e as 4 notas de 50 alunos de uma escola, e depois **listar** o nome de cada aluno junto com sua média.



```
Digite o nome do aluno: Paulo
  Digite a nota 1 do aluno :50
  Digite a nota 2 do aluno :80
  Digite a nota 3 do aluno :40
  Digite a nota 4 do aluno :60
Digite o nome do aluno: José
  Digite a nota 1 do aluno :80
  Digite a nota 2 do aluno :100
  Digite a nota 3 do aluno :73
  Digite a nota 4 do aluno :70
Digite o nome do aluno:
```

```
**** Alunos - Medias****
```

```
Paulo - 57.5
```

```
José - 80.75
```

...

...

VETORES

- Em casos como esse que é útil a utilização da **estrutura de dados** conhecida como **vetor**
- Um vetor é uma espécie de caixa com várias divisórias para armazenar coisas (dados)
 - É uma variável que pode armazenar vários valores

VETORES

meuVetor

--	--	--	--	--	--	--	--

medias

10	40	8	26	70	73		
----	----	---	----	----	----	--	--

nomes

Paulo	José	Maria	Ricardo				
-------	------	-------	---------	--	--	--	--

SINTAXE NO VISUALG

○ Preenchendo um vetor

- Podemos utilizar um laço de repetição para facilitar o preenchimento dos dados em vetores

○ Exemplo:

```
algoritmo "exemplo_vetores"
var
    numeros: vetor [1..10] de inteiro
    i: inteiro
inicio
    para i de 1 ate 10 faca
        escreva("Digite um valor para ser adicionado ao vetor")
        leia(numeros[i])
    fimpara
fimpara
```

SINTAXE NO VISUALG

○ Preenchendo um vetor

```
algoritmo "exemplo_vetores"
var
    numeros: vetor [1..5] de inteiro
inicio
    escreva("Digite um valor para a posição 1 do vetor:")
    leia(numeros[1])
    escreva("Digite um valor para a posição 2 do vetor:")
    leia(numeros[2])
    escreva("Digite um valor para a posição 3 do vetor:")
    leia(numeros[3])
    escreva("Digite um valor para a posição 4 do vetor:")
    leia(numeros[4])
    escreva("Digite um valor para a posição 5 do vetor:")
    leia(numeros[5])
fimpara
```


SINTAXE NO VISUALG

○ Preenchendo um vetor

- Para facilitar, podemos utilizar um laço de repetição!

○ Exemplo:

```
algoritmo "exemplo_vetores"
var
    numeros: vetor [1..5] de inteiro
    i: inteiro
inicio
    para i de 1 ate 5 faca
        escreva("Digite um valor para a posição ", i , "do vetor:")
        leia(numeros[i])
    fimpara
fimpara
```

SINTAXE NO VISUALG

○ Exibindo o conteúdo de um vetor:

```
...  
    escreva("O valor que está na posição 1 é: ", numeros[1])  
    escreva("O valor que está na posição 2 é: ", numeros[2])  
    escreva("O valor que está na posição 3 é: ", numeros[3])  
    escreva("O valor que está na posição 4 é: ", numeros[4])  
    escreva("O valor que está na posição 5 é: ", numeros[5])  
fimalgoritmo
```

SINTAXE NO VISUALG

○ Exibindo o conteúdo de um vetor

- Ou podemos utilizar um laço de repetição para facilitar a exibição dos valores de um vetor

○ Exemplo:

```
para i de 1 ate 5 faca
    escreva("O valor que está na posição ", i , " é: ", numeros[i])
fimpara
```

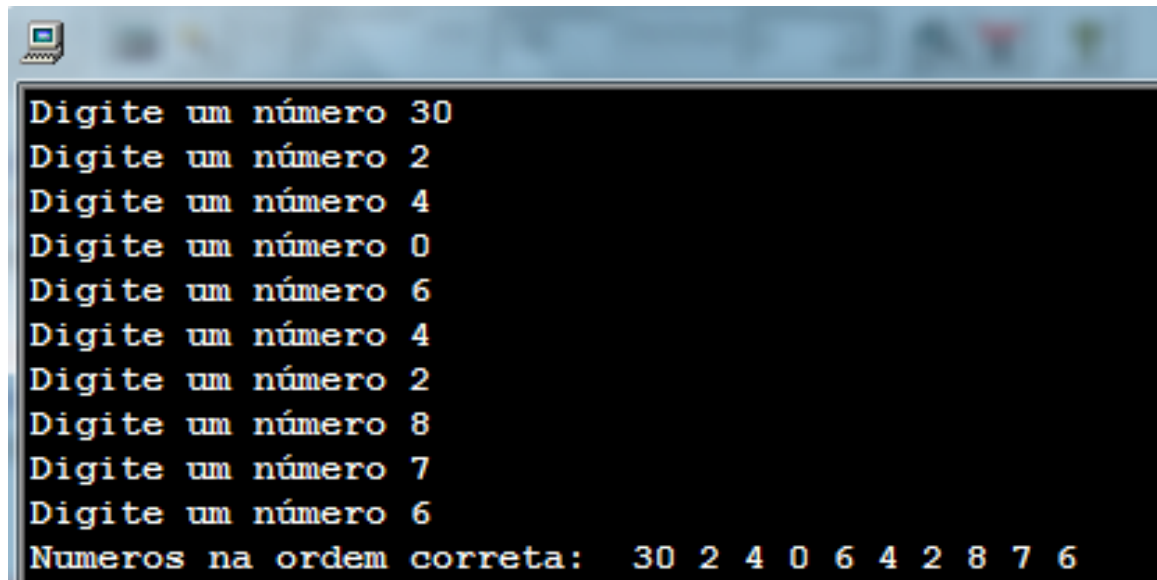
EXEMPLO 1

- Criar um algoritmo que leia 10 números pelo teclado e exiba os números na ordem correta que os números foram digitados.

```
algoritmo "vetor"  
var  
    numeros: vetor [1..10] de inteiro  
    i: inteiro  
inicio  
    para i de 1 ate 10 faca  
        escreva("Digite um número ")  
        leia(numeros[i])  
    fimpara  
    escreva("Numeros na ordem correta: ")  
    para i de 1 ate 10 faca  
        escreva(numeros[i])  
    fimpara  
fimalgoritmo
```

EXEMPLO 1

- Saída:

A screenshot of a terminal window with a black background and white text. The text shows a sequence of prompts "Digite um número" followed by the input of numbers: 30, 2, 4, 0, 6, 4, 2, 8, 7, 6. The final line displays the sorted sequence: "Numeros na ordem correta: 30 2 4 0 6 4 2 8 7 6".

```
Digite um número 30
Digite um número 2
Digite um número 4
Digite um número 0
Digite um número 6
Digite um número 4
Digite um número 2
Digite um número 8
Digite um número 7
Digite um número 6
Numeros na ordem correta: 30 2 4 0 6 4 2 8 7 6
```

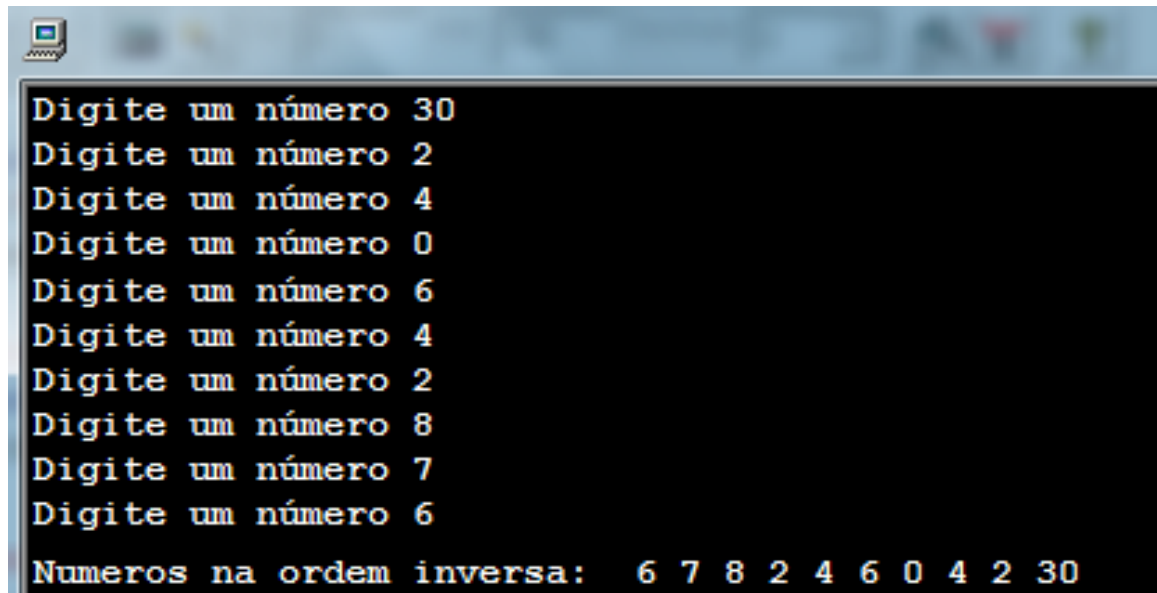
EXEMPLO 2

- Criar um algoritmo que leia 10 números pelo teclado e exiba os números na ordem inversa da que os números foram digitados.

```
algoritmo "vetor"  
var  
    numeros: vetor [1..10] de inteiro  
    i: inteiro  
inicio  
    para i de 1 ate 10 faca  
        escreva("Digite um número ")  
        leia(numeros[i])  
    fimpara  
    escreva("Numeros na ordem inversa: ")  
    para i de 10 ate 1 passo -1 faca  
        escreva(numeros[i])  
    fimpara  
fimalgoritmo
```

EXEMPLO 2

- Saída:



```
Digite um número 30
Digite um número 2
Digite um número 4
Digite um número 0
Digite um número 6
Digite um número 4
Digite um número 2
Digite um número 8
Digite um número 7
Digite um número 6
Numeros na ordem inversa:  6 7 8 2 4 6 0 4 2 30
```

EXEMPLO 3

- Escreva um algoritmo que leia um vetor com 10 posições de números inteiros. Em seguida, receba um novo valor do usuário e verifique se este valor se encontra no vetor.

EXERCÍCIOS

1. Crie um algoritmo que leia um vetor de 10 números inteiros. Em seguida, calcule e escreva o somatório dos valores deste vetor.
2. Escreva um algoritmo que leia um vetor com 15 posições de números inteiros. Em seguida, escreva somente os números positivos que se encontram no vetor.
3. Escreva um algoritmo que leia um vetor com 8 posições de números inteiros. Em seguida, leia um novo valor do usuário e verifique se o valor se encontra no vetor. Se estiver, informe a posição desse elemento no vetor. Caso o elemento não esteja no vetor, apresente uma mensagem informando “O número não se encontra no vetor”.

EXERCÍCIOS

5. Escreva um algoritmo que leia **dois** vetores de 10 posições e faça a soma dos elementos de mesmo índice, colocando o resultado em um terceiro vetor. Mostre o vetor resultante.

Exemplo:

vetor 1	7	4	9	15	20	2	1	4	0	30
vetor 2	1	8	3	7	14	9	1	8	11	16
vetorResultado	8	12	12	22	34	11	2	12	11	46

6. Crie um algoritmo que leia um vetor de 20 posições e informe:
- a) Quantos números pares existem no vetor
 - b) Quantos números ímpares existem no vetor
 - c) Quantos números maiores do que 50
 - d) Quantos números menores do que 7

Vetores

- ▶ Até agora: **variável e constante**
- ▶ Armazenar centenas de valores -> criar centenas de variáveis
 - ▶ Não é uma boa solução
- ▶ Ex: armazenar as notas dos alunos
 - ▶ Cálculo da média
 - ▶ Ordenar as notas

Exemplo

- A média aritmética de um conjunto de valores é dada pela seguinte expressão:

$$m = \frac{\sum_{i=1}^n x_i}{n}$$

- O programa abaixo calcula a média aritmética entre cinco valores:

```
int main()
{
    float nota1, nota2, nota3, nota4, nota5;
    printf("Entre com a 1a. nota:");
    scanf("%f", &nota1);
    printf("Entre com a 2a. nota:");
    scanf("%f", &nota2);
    printf("Entre com a 3a. nota:");
    scanf("%f", &nota3);
    printf("Entre com a 4a. nota:");
    scanf("%f", &nota4);
    printf("Entre com a 5a. nota:");
    scanf("%f", &nota5);
    printf("Média = %f", (nota1+nota2+nota3+nota4+nota5)/5);
}
```

Como seria este programa se precisarmos calcular a média entre 200 notas?

Solução: Variáveis Compostas

- ▶ Conjunto (coleção) de valores de um mesmo tipo de dados
- ▶ Podem ser:
 - ▶ Unidimensionais: vetores
 - ▶ Multidimensionais: matrizes

Variáveis Compostas

Unidimensionais: Vetores (Arrays)

- ▶ Tipo de dado usado para representar uma coleção de variáveis de um mesmo tipo.
- ▶ Estrutura de dados homogênea e unidimensional.
- ▶ Sintaxe:

tipo nome_do_vetor[tamanho];

- ▶ Tamanho representa o número de elementos;
- ▶ O índice do vetor varia de 0 a (tamanho - 1);
- ▶ Ex.:

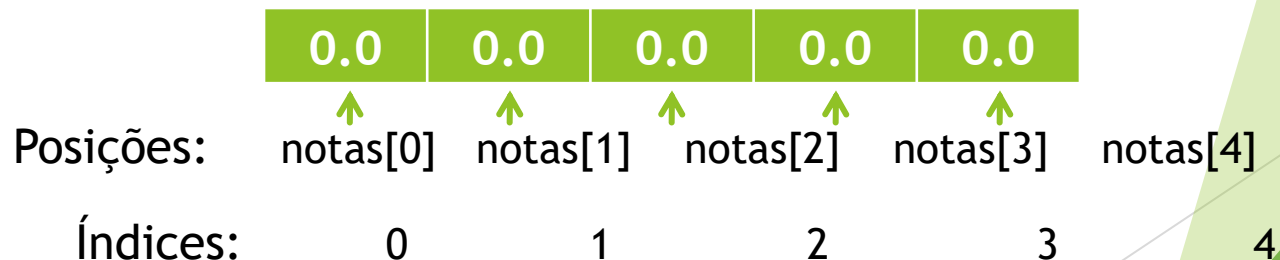
```
int main()  
{  
    float notas[5];  
    ...  
}
```

Declara um vetor do tipo **float** com 5 posições



Vetores (Arrays)

- ▶ Coleção de caixas de variáveis
- ▶ Variáveis alocadas sequencialmente na memória
- ▶ Endereço inicial corresponde ao primeiro elemento (índice 0) do vetor.
- ▶ O acesso a cada posição do vetor realizado utilizando-se seu índice:



Acesso aos Elementos

- ▶ Nome do vetor seguido do índice do elemento entre colchetes
- ▶ **notas[2]** é 3º elemento do array:



- ▶ **notas[2]** é uma variável do tipo float

```
int main()  
{  
    float notas[5];  
    ...  
}
```


Exemplo

- Ler a nota de 5 alunos de uma disciplina e calcular a média.

```
int main()
{
    float nota[5];
    float soma = 0;
    int i;

    for (i = 0; i < 5; i++) {
        printf("Entre com a %da. nota: ", (i + 1));
        scanf("%f", &nota[i]);
        soma = soma + nota[i];
    }
    printf("Média da disciplina = %f", soma/5);
}
```

Declara um vetor de 5 posições.

Armazena o valor lido na i-ésima posição.

Acessa o valor da i-ésima posição.

Vetores: Valores nos Colchetes

► Significados diferentes:

- Na **declaração** de um vetor: informa a quantidade N de posições que devem ser alocadas
- Após a declaração: informa a posições que será acessada para leitura ou gravação de informação. Deve ser um valor

```
int main()
{
    float nota[5];
    float soma = 0;
    int i;

    for (i = 0; i < 5; i++) {
        printf("Entre com a %da. nota: ", (i + 1));
        scanf("%f", &nota[i]);
        soma = soma + nota[i];
    }
    printf("Média da disciplina = %f", soma/5);
}
```

Declara um vetor de $N=5$ posições.

Grava informação na posição i em $[0...4]$.

Lê informação da posição i em $[0...4]$.

Vetores e seus limites

```
int main()  
{  
    float notas[5];  
    ...  
}
```

- ▶ No exemplo anterior:
 - ▶ Vetor possui cinco posições
 - ▶ Índice da última posição é 4
 - ▶ Acessar **notas[5]** causa falha de memória.
- ▶ Índice deve variar de 0 a [tamanho - 1];
- ▶ C não avisa quando o limite de um vetor é excedido!
- ▶ Se o programa transpuser o fim do vetor durante a operação de atribuição, os valores serão armazenados em posições inválidas de memória, ou sobrescrevendo outras variáveis;

O programador tem a responsabilidade de verificar o limite do vetor!

Vetores e seus limites

- Erro comum: acesso fora dos limites

```
int main() {  
    int pares[20];  
    int i, somaPares = 0;  
    for (i = 0; i <= 20; i++) {  
        pares[i] = 2 * i;  
        somaPares = somaPares + pares[i];  
    }  
    ...  
}
```

Deveria ser: $i < 20$

A posição `pares[20]` está fora do vetor!

Onde está o erro?

Vetores: Inicialização

► Tipo 1:

```
int v[5] = {5,10,15,20,25};
```

► Tipo 2:

```
int v[] = {5,10,15,20,25};
```

- Compilador aloca espaço suficiente para armazenar todos os valores

Vetores só podem ser inicializados dessa forma em sua declaração!

Vetores: Inicialização

- ▶ Quando o tamanho do vetor for especificado e houver a lista de inicialização:
 - ▶ Se há menos inicializadores que o tamanho especificado, os outros serão zero;
 - ▶ Mais inicializadores que o necessário implica em um aviso de compilação (*warning*).
- ▶ Quando não inicializado: o tamanho deve ser especificado na declaração

Vetores: Declaração do Tamanho

- ▶ Valor literal, não uma variável
 - ▶ Determinado em tempo de compilação

▶ Ex

```
int nAlunos = 10;
printf ("entre com o número de alunos");
scanf ("%d", &nAlunos);
int notas[nAlunos];
...
```

Erro!
nAlunos é
desconhecido em
tempo de compilação

Vetores: Constantes #define pro Tamanho

► Conhecidas em tempo de compilação

```
#include <stdio.h>
#define TOTAL_ALUNOS 10

int main()
{
    float nota[TOTAL_ALUNOS];
    float soma = 0;
    int i;

    for (i = 0; i < TOTAL_ALUNOS; i++) {
        printf("Entre com a %da. nota: ", (i + 1));
        scanf("%f", &nota[i]);
        soma = soma + nota[i];
    }
    printf("Média da disciplina = %f", soma/TOTAL_ALUNOS);
}
```

Recomendável! pois facilita a legibilidade e a manutenção do código. Se for preciso aumentar o número de alunos basta modificá-lo em um local.

Vetores como parâmetro de funções

- ▶ Pode ser passado como argumento para uma função
- ▶ Ao passar um vetor para uma função podemos modificar o conteúdo deste vetor dentro da função:
 - ▶ Passa-se na verdade o endereço do primeiro elemento do vetor na memória;
 - ▶ Os demais estão nas posições seguintes de memória;
- ▶ Podemos passar também um elemento em particular de um vetor para uma função
 - ▶ O parâmetro deve ser do mesmo tipo do vetor

Vetores como parâmetro de funções

```
#include <stdio.h>

float media(int n, float num[])
{
    int i;
    float s = 0.0;
    for(i = 0; i < n; i++)
        s = s + num[i] ;
    return s/n ;
}

int main()
{
    float numeros[10] ;
    float med;
    int i ;
    for(i = 0; i < 10; i++)
        scanf ("%f", &numeros[i]) ;
    med = media(10, numeros) ;
    ...
}
```

Recebe um vetor float e o número de elementos como parâmetro e calcula a média

Declaração de um vetor float de 10 posições

Passa este vetor como argumento para a função “media”

Vetores como parâmetro de funções

```
#include <stdio.h>

void incrementar(float num[], int n, float valor)
{
    int i;
    for(i = 0; i < n; i++)
        num[i] = num[i] + valor;
}

int main()
{
    float numeros[10];
    int i;
    for(i = 0; i < 10; i++)
        scanf("%f", &numeros[i]);

    incrementar(numeros, 10, 1.5);

    for(i = 0; i < 10; i++)
        printf("%f ", numeros[i]);

    return 0;
}
```

Recebe um vetor float, o número de elementos e um valor que será utilizado para modificar o vetor

Declaração de um vetor float de 10 posições

Passa este vetor juntamente com os outros valores como argumento para a função “incrementar”

Imprime o vetor modificado por “incrementar”

Posição de vetor como parâmetro de funções

```
#include <stdio.h>

int aprovado(float nota, float media)
{
    int resultado = 0;
    if ( nota >= media)
        resultado = 1;
    return resultado;
}

int main()
{
    float notas[] = {6.5, 5.0, 7.5, 9.4, 3.8};
    int i ;
    for(i = 0; i < 5; i++)
    {
        if (aprovado(notas[i], 7.0) == 1)
            printf("Aluno %d: aprovado\n", i);
        else
            printf("Aluno %d: REPROVADO\n", i);
    }
    ...
}
```

Recebe uma nota float, e a média com a qual a nota será comparada

Declaração de um vetor com as notas já preenchidas

Passa o valor na posição i do vetor como argumento para a função “aprovado”

MATRIZES

- O que é uma matriz?
 - Uma estrutura de dados que contém várias variáveis do mesmo tipo
- Qual a diferença de **vetores** para **matrizes**?
 - Vetores são, na verdade, matrizes de uma única dimensão:

Vetores

1	3	4	6
---	---	---	---

a	maria	jota
---	-------	------

Matrizes

1	3
40	4
6	12

M	J	K
G	A	C
L	Z	H

1.1	7.5	9.2	8.8
9.0	1.3	5.5	7.9

MATRIZES

- As matrizes são, comumente referenciadas através de suas dimensões (quantidade de linhas e colunas)
- Anotação comum é: $M \times N$, onde
 - M é a dimensão vertical (**quantidade de linhas**)
 - N é dimensão horizontal (**quantidade de colunas**)
- Exemplo:

3x3

3x2

2x3

4x1

1x3

--	--	--

Vetores: a quantidade de linhas é sempre 1!

MATRIZES

○ Notação

- Como referenciar um elemento específico da matriz?
- Exemplo: Matriz 3x2 (*três linhas e duas colunas*)

A diagram illustrating a 3x2 matrix. The matrix is represented as a 3x2 grid of empty cells. To the left of the grid, a vertical green bracket groups the rows, with the numbers 1, 2, and 3 listed next to it. Above the grid, a horizontal green bracket groups the columns, with the numbers 1 and 2 listed above it.

As linhas
variam de 1
até 3

As colunas
variam de 1
até 2

MATRIZES

○ Notação

- Como referenciar um elemento específico da matriz?
- Exemplo: Matriz 3x2 (*três linhas e duas colunas*)

Para acessar esse elemento, devemos observar qual cruzamento linha x coluna da matriz ele representa

	1	2
1	1,1	
2		
3		

Linha 1
Coluna 1

MATRIZES

○ Notação

- Como referenciar um elemento específico da matriz?
- Exemplo: Matriz 3x2 (*três linhas e duas colunas*)

	1	2
1	1,1	1,2
2		
3		

Linha 1
Coluna 2

MATRIZES

○ Notação

- Como referenciar um elemento específico da matriz?
- Exemplo: Matriz 3x2 (*três linhas e duas colunas*)

	1	2
1	1,1	1,2
2	2,1	
3		

Linha 2
Coluna 1

MATRIZES

○ Notação

- Como referenciar um elemento específico da matriz?
- Exemplo: Matriz 3x2 (*três linhas e duas colunas*)

	1	2
1	1,1	1,2
2	2,1	2,2
3		

Linha 2
Coluna 2

MATRIZES

○ Notação

- Como referenciar um elemento específico da matriz?
- Exemplo: Matriz 3x2 (*três linhas e duas colunas*)

	1	2
1	1,1	1,2
2	2,1	2,2
3	3,1	3,2

Diagram illustrating a 3x2 matrix with row and column indices. The matrix is shown with rows labeled 1, 2, 3 and columns labeled 1, 2. The elements are labeled as (row, column). Two callout boxes highlight specific elements:

- Linha 3 Coluna 1 (points to element 3,1)
- Linha 3 Coluna 2 (points to element 3,2)

SINTAXE NO VISUALG

○ Declaração:

```
<nome_variavel>: vetor [li..lf, ci..cf] de <tipo>
```

○ Onde:

- li e lf representam, respectivamente o índice inicial e final das **linhas** e
- ci e cf representam, respectivamente o índice inicial e final das **colunas**

SINTAXE NO VISUALG

○ Exemplo:

- Para declarar uma matriz 3x2 de inteiro

```
algoritmo "exemplo_matriz"  
var  
    exMatriz: vetor [1..3, 1..2] de inteiro  
inicio  
    ...
```

Linhas: o índice das
linhas varia de 1 até 3

Colunas: o índice das
colunas varia de 1 até 2

SINTAXE NO VISUALG

○ **Preenchendo e acessando uma matriz**

- As posições das matrizes são identificados pelos índices das linhas e colunas

○ **Atribuição**

```
<nome_variavel> [<linha>, <coluna>] ← <valor>  
<nome_variavel> [<linha>, <coluna>] := <valor>  
leia(<nome_variavel> [<linha>, <coluna>])
```

SINTAXE NO VISUALG

o Exemplo:

```
algoritmo "exemplo_matriz"
```

```
var
```

```
exMatriz: vetor [1..3, 1..2] de inteiro
```

```
inicio
```

```
exMatriz[1,1] ← 10
```

```
leia(exMatriz[1,2])
```

```
exMatriz[3,1] := 4
```

```
fimalgoritmo
```

	1	2
1		
2		
3		

exMatriz

SINTAXE NO VISUALG

o Exemplo:

```
algoritmo "exemplo_matriz"  
var  
    exMatriz: vetor [1..3, 1..2] de inteiro  
inicio  
    exMatriz[1,1] ← 10  
    leia(exMatriz[1,2])  
    exMatriz[3,1] := 4  
fimalgoritmo
```

	1	2
1	10	
2		
3		

exMatriz

SINTAXE NO VISUALG

o Exemplo:

```
algoritmo "exemplo_matriz"  
var  
    exMatriz: vetor [1..3, 1..2] de inteiro  
inicio  
    exMatriz[1,1] ← 10  
    leia(exMatriz[1,2])  
    exMatriz[3,1] := 4  
fimalgoritmo
```

	1	2
1	10	7
2		
3		

exMatriz

SINTAXE NO VISUALG

o Exemplo:

```
algoritmo "exemplo_matriz"  
var  
    exMatriz: vetor [1..3, 1..2] de inteiro  
inicio  
    exMatriz[1,1] ← 10  
    leia(exMatriz[1,2])  
    exMatriz[3,1] := 4  
fimalgoritmo
```

	1	2
1	10	7
2		
3	4	

exMatriz

SINTAXE NO VISUALG

○ Preenchendo uma matriz

- Se quisermos atribuir valores a todas as posições da matriz, podemos fazer:

```
algoritmo "preencher_matrizes"
var
    numeros: vetor[1..3, 1..2] de inteiro
    i: inteiro
inicio
    para i de 1 ate 3 faca //fazer o laço para as linhas
        escreva("Digite o valor para a posicao ", i, ", 1:")
        leia(numeros[i, 1])
        escreva("Digite o valor para a posicao ", i, ", 2:")
        leia(numeros[i, 2])
    fimpara
finalgoritmo
```

SINTAXE NO VISUALG

○ Preenchendo uma matriz

- Se quisermos atribuir valores a todas as posições da matriz, podemos fazer:

```
algoritmo "preencher"  
var  
    numeros: vetor[1..3, 1..2] de inteiro  
    i, j: inteiro  
inicio  
    escreva("Digite um valor para a posição [1,1]")  
    leia(numeros[1,1])  
    escreva("Digite um valor para a posição [1,2]")  
    leia(numeros[1,2])  
    escreva("Digite um valor para a posição [2,1]")  
    leia(numeros[2,1])  
    escreva("Digite um valor para a posição [2,2]")  
    leia(numeros[2,2])  
    escreva("Digite um valor para a posição [3,1]")  
    leia(numeros[3,1])  
    escreva("Digite um valor para a posição [3,2]")  
    leia(numeros[3,2])  
fimalgoritmo
```

SINTAXE NO VISUALG

○ **Preenchendo uma matriz**

- Entretanto, à medida que a quantidade de elementos da matriz aumenta, fica complicado fazermos manualmente para todas as posições.
- O melhor caminho é utilizar laços de repetição!

SINTAXE NO VISUALG

○ Preenchendo uma matriz

- Podemos criar um laço de repetição para variar pelas linhas, por exemplo:

```
algoritmo "preencher"  
var  
    numeros: vetor[1..3, 1..2] de inteiro  
    i, j: inteiro  
inicio  
    para i de 1 ate 3 faca  
        escreva("Digite um valor para a posição [", i, ",1")  
        leia(numeros[i,1])  
        escreva("Digite um valor para a posição [", i, ",2")  
        leia(numeros[i,2])  
    fimpara  
fimalgoritmo
```

SINTAXE NO VISUALG

○ Preenchendo uma matriz

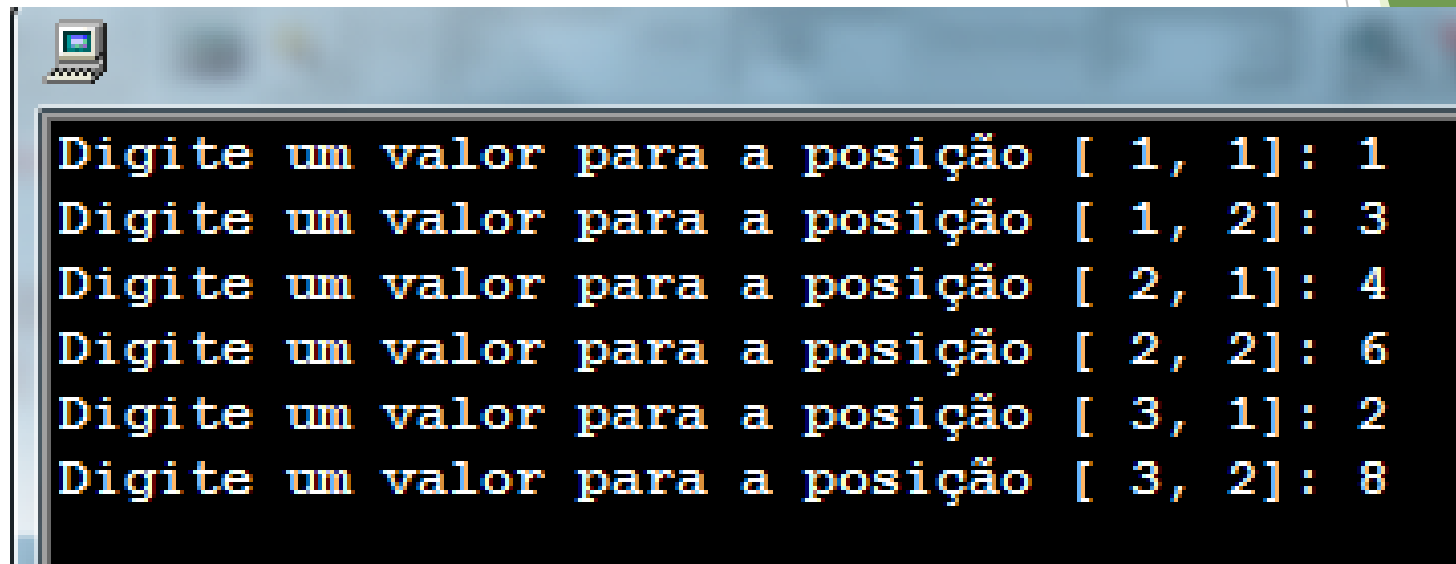
- E podemos ainda incluir um laço de repetição para variar pelas colunas também, por exemplo:

```
algoritmo "preencher"  
var  
    numeros: vetor[1..3, 1..2] de inteiro  
    i, j: inteiro  
inicio  
    para i de 1 ate 3 faca  
        para j de 1 ate 2 faca  
            escreva("Digite um valor para a posição [", i, ",", j, "]: ")  
            leia(numeros[i,j])  
        fimpara  
    fimpara  
fimalgoritmo
```


SINTAXE NO VISUALG

- **Preenchendo uma matriz**

- Saída:



```
Digite um valor para a posição [ 1, 1]: 1
Digite um valor para a posição [ 1, 2]: 3
Digite um valor para a posição [ 2, 1]: 4
Digite um valor para a posição [ 2, 2]: 6
Digite um valor para a posição [ 3, 1]: 2
Digite um valor para a posição [ 3, 2]: 8
```

SINTAXE NO VISUALG

○ Exibindo o conteúdo de uma matriz:

```
...
```

```
    escreva("O valor que está na posição [1,1] é: ", numeros[1,1])  
    escreva("O valor que está na posição [1,2] é: ", numeros[1,2])  
    escreva("O valor que está na posição [2,1] é: ", numeros[2,1])  
    escreva("O valor que está na posição [2,2] é: ", numeros[2,2])  
    escreva("O valor que está na posição [3,1] é: ", numeros[3,1])  
    escreva("O valor que está na posição [3,2] é: ", numeros[3,2])
```

```
fimalgoritmo
```

SINTAXE NO VISUALG

○ Exibindo o conteúdo de uma matriz

- Ou podemos utilizar um laço de repetição para facilitar a exibição dos valores de uma matriz
- Criando um laço para percorrer as linhas:

○ Exemplo:

```
para i de 1 ate 3 faca
    escreval("O valor que está na posição [", i, ", 1] é: ", numeros[i, 1])
    escreval("O valor que está na posição [", i, ", 2] é: ", numeros[i, 2])
fimpara
```

SINTAXE NO VISUALG

○ Exibindo o conteúdo de uma matriz

- E podemos ainda incluir um laço de repetição para variar pelas colunas também, por exemplo:

```
para i de 1 ate 3 faca
    para j de 1 ate 2 faca
        escreval("O valor que está na posição [", i,",", j,"] é: ", numeros[i, j])
    fimpara
fimpara
```

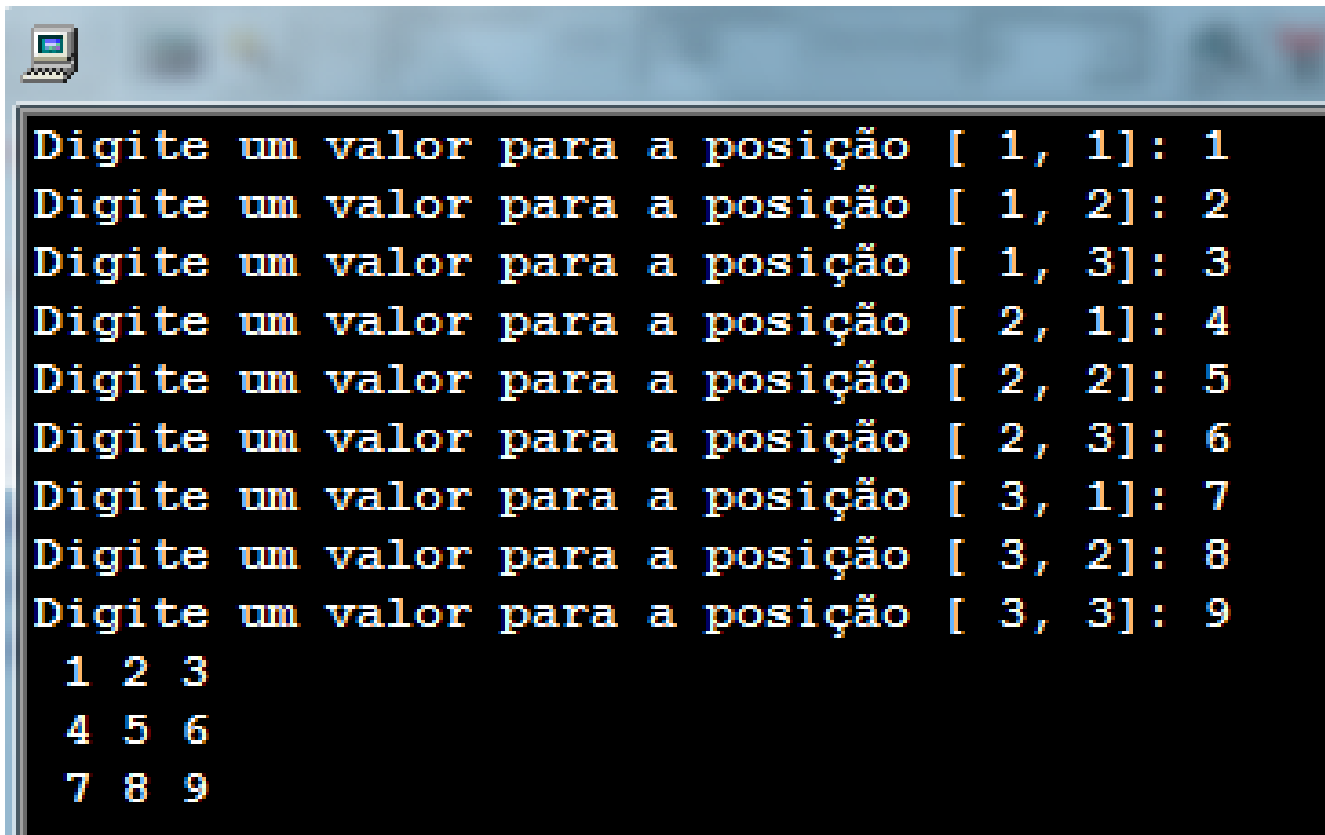
EXEMPLO 1

- Criar um algoritmo que leia uma matriz 3x3 e exiba a matriz preenchida:

```
algoritmo "exemplo01"  
var  
  numeros: vetor[1..3, 1..3] de inteiro  
  i, j: inteiro  
inicio  
  para i de 1 ate 3 faca  
    para j de 1 ate 3 faca  
      escreva("Digite um valor para a posição [", i, ",", j, "]: ")  
      leia(numeros[i,j])  
    fimpara  
  fimpara  
  
  para i de 1 ate 3 faca  
    para j de 1 ate 3 faca  
      escreva(numeros[i, j])  
    fimpara  
  escreval  
fimpara  
fimalgoritmo
```

EXEMPLO 1

- Saída:



```
Digite um valor para a posição [ 1, 1]: 1
Digite um valor para a posição [ 1, 2]: 2
Digite um valor para a posição [ 1, 3]: 3
Digite um valor para a posição [ 2, 1]: 4
Digite um valor para a posição [ 2, 2]: 5
Digite um valor para a posição [ 2, 3]: 6
Digite um valor para a posição [ 3, 1]: 7
Digite um valor para a posição [ 3, 2]: 8
Digite um valor para a posição [ 3, 3]: 9

1 2 3
4 5 6
7 8 9
```

The image shows a terminal window with a dark background and light-colored text. It displays a sequence of prompts asking for values for a 3x3 matrix. The prompts are: "Digite um valor para a posição [1, 1]:", "Digite um valor para a posição [1, 2]:", "Digite um valor para a posição [1, 3]:", "Digite um valor para a posição [2, 1]:", "Digite um valor para a posição [2, 2]:", "Digite um valor para a posição [2, 3]:", "Digite um valor para a posição [3, 1]:", "Digite um valor para a posição [3, 2]:", and "Digite um valor para a posição [3, 3]:". The values entered are 1, 2, 3, 4, 5, 6, 7, 8, and 9 respectively. Below the prompts, the resulting 3x3 matrix is displayed as follows:

1	2	3
4	5	6
7	8	9

EXEMPLO 2

- Criar um algoritmo que leia uma matrizes 3x3. Em seguida, exiba a som dos elementos de cada uma das linhas. Ex:

1	2	2
3	2	3
4	1	1

Soma Linha 1 = 5

Soma Linha 2 = 8

Soma Linha 3 = 6

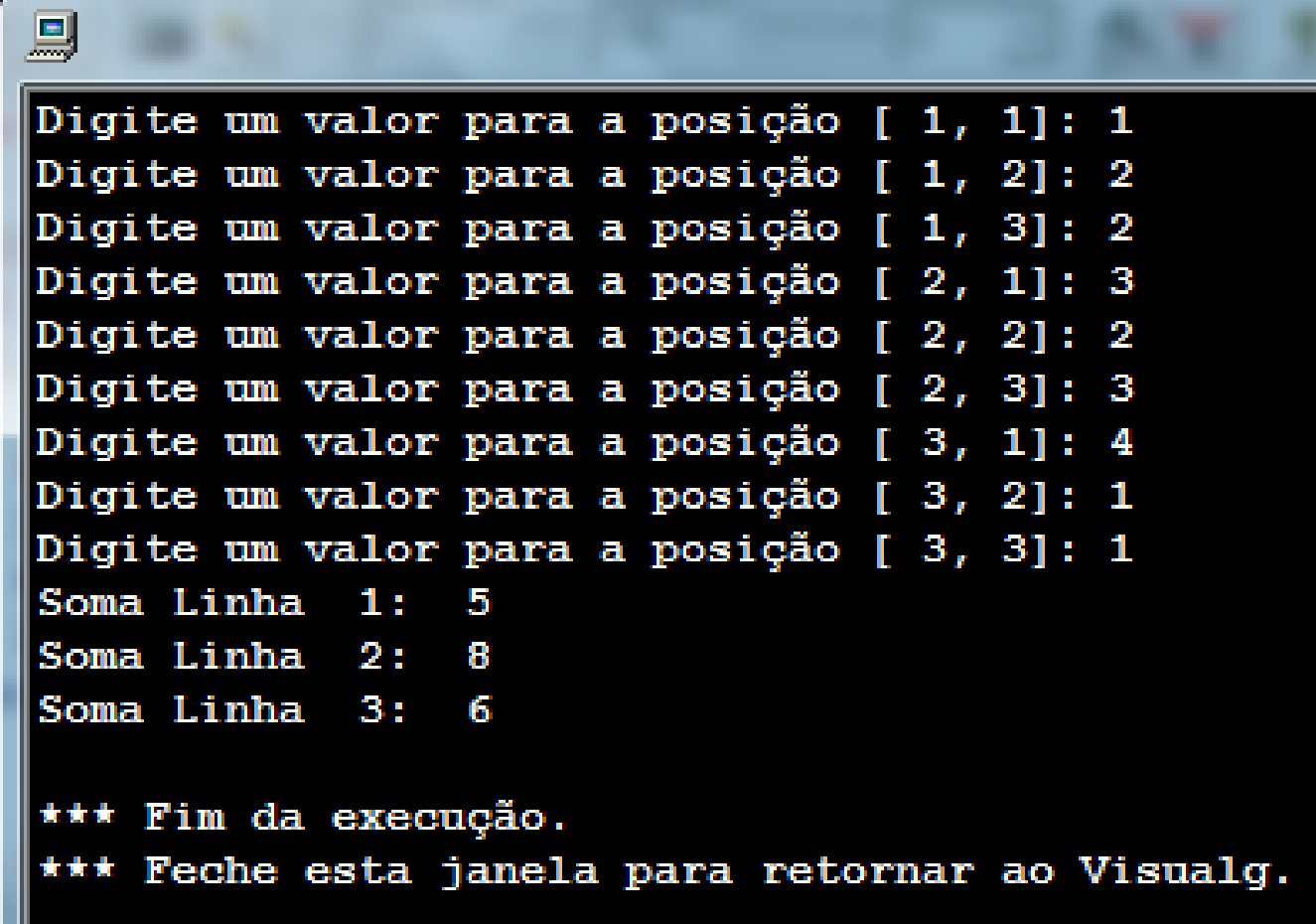
EXEMPLO 2

○ Resolução:

```
algoritmo "exemplo01"  
var  
    numeros: vetor[1..3, 1..3] de inteiro  
    i, j: inteiro  
    soma: inteiro  
inicio  
    para i de 1 ate 3 faca  
        para j de 1 ate 3 faca  
            escreva("Digite um valor para a posição [", i, ",", j, "]: ")  
            leia(numeros[i,j])  
        fimpara  
    fimpara  
  
    para i de 1 ate 3 faca  
        soma <- 0  
        para j de 1 ate 3 faca  
            soma <- soma + numeros[i, j]  
        fimpara  
        escreval ("Soma Linha ", i, ": ", soma)  
    fimpara  
fimalgoritmo
```


EXEMPLO 2

- Saída:



```
Digite um valor para a posição [ 1, 1]: 1
Digite um valor para a posição [ 1, 2]: 2
Digite um valor para a posição [ 1, 3]: 2
Digite um valor para a posição [ 2, 1]: 3
Digite um valor para a posição [ 2, 2]: 2
Digite um valor para a posição [ 2, 3]: 3
Digite um valor para a posição [ 3, 1]: 4
Digite um valor para a posição [ 3, 2]: 1
Digite um valor para a posição [ 3, 3]: 1
Soma Linha  1:  5
Soma Linha  2:  8
Soma Linha  3:  6

*** Fim da execução.
*** Feche esta janela para retornar ao Visualg.
```

EXEMPLO 3

- Escreva um algoritmo que leia uma matriz 4x3. Em seguida, receba um novo valor do usuário e verifique se este valor se encontra na matriz. Caso o valor se encontre na matriz, escreva a mensagem “O valor se encontra na matriz”. Caso contrário, escreva a mensagem “O valor NÃO se encontra na matriz”.

EXEMPLO 3

algoritmo "exemplo03"

var

numeros: vetor[1..4, 1..3] de inteiro

i, j, buscar: inteiro

achou: logico

inicio

para i de 1 ate 4 faca

para j de 1 ate 3 faca

escreva("Digite um valor para a posição [", i, ",", j, "]: ")

leia(numeros[i,j])

fimpara

fimpara

escreva("Digite um valor para ser buscado na matriz: ")

leia(buscar)

achou <- falso

para i de 1 ate 4 faca

para j de 1 ate 3 faca

se (numeros[i,j] = buscar) entao

achou <- verdadeiro

fimse

fimpara

fimpara

se achou=verdadeiro entao

escreva("O número se encontra na matriz.")

senao

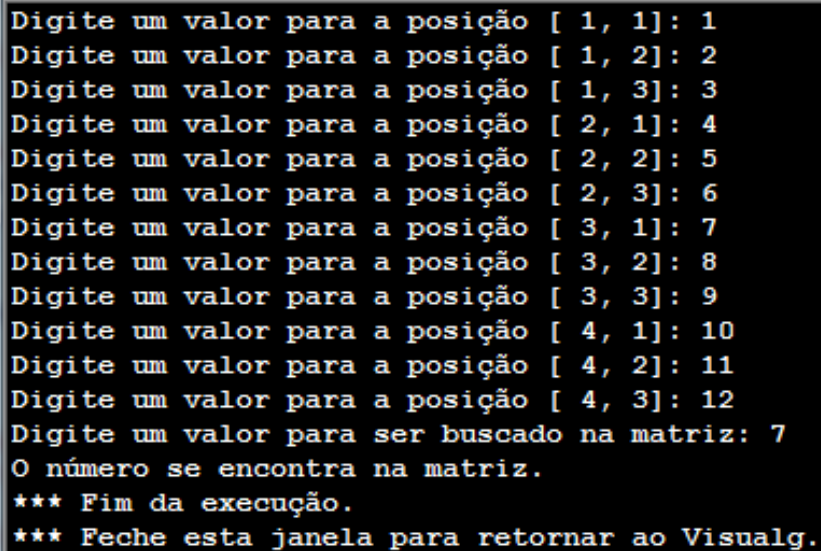
escreva("O número NÃO se encontra na matriz.")

fimse

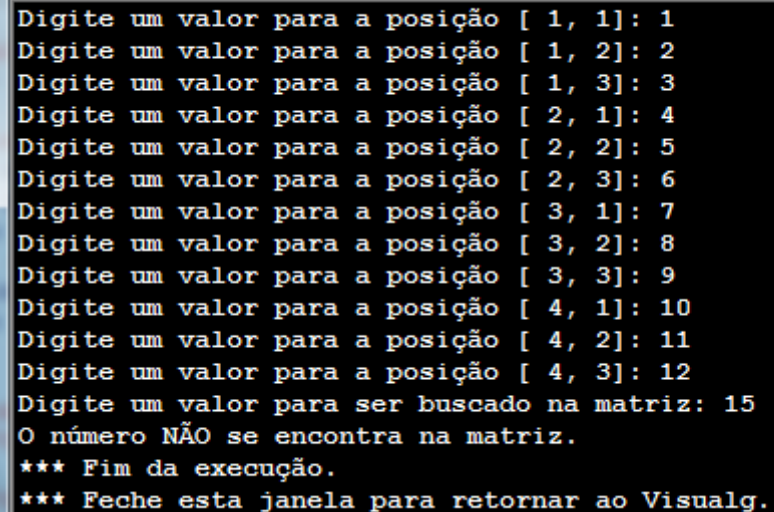
fimalgoritmo

EXEMPLO 3

- Saída:



```
Digite um valor para a posição [ 1, 1]: 1
Digite um valor para a posição [ 1, 2]: 2
Digite um valor para a posição [ 1, 3]: 3
Digite um valor para a posição [ 2, 1]: 4
Digite um valor para a posição [ 2, 2]: 5
Digite um valor para a posição [ 2, 3]: 6
Digite um valor para a posição [ 3, 1]: 7
Digite um valor para a posição [ 3, 2]: 8
Digite um valor para a posição [ 3, 3]: 9
Digite um valor para a posição [ 4, 1]: 10
Digite um valor para a posição [ 4, 2]: 11
Digite um valor para a posição [ 4, 3]: 12
Digite um valor para ser buscado na matriz: 7
O número se encontra na matriz.
*** Fim da execução.
*** Feche esta janela para retornar ao Visualg.
```



```
Digite um valor para a posição [ 1, 1]: 1
Digite um valor para a posição [ 1, 2]: 2
Digite um valor para a posição [ 1, 3]: 3
Digite um valor para a posição [ 2, 1]: 4
Digite um valor para a posição [ 2, 2]: 5
Digite um valor para a posição [ 2, 3]: 6
Digite um valor para a posição [ 3, 1]: 7
Digite um valor para a posição [ 3, 2]: 8
Digite um valor para a posição [ 3, 3]: 9
Digite um valor para a posição [ 4, 1]: 10
Digite um valor para a posição [ 4, 2]: 11
Digite um valor para a posição [ 4, 3]: 12
Digite um valor para ser buscado na matriz: 15
O número NÃO se encontra na matriz.
*** Fim da execução.
*** Feche esta janela para retornar ao Visualg.
```

EXERCÍCIOS

1. Crie um algoritmo que leia uma matriz 5x5. Em seguida, conte quantos números pares existem na matriz.
2. Crie um algoritmo que leia uma matriz 3x3 e calcule a soma dos valores das colunas da matriz. Ex:

1	2	2
3	2	3
4	1	1

Soma Coluna 1 = 8

Soma Coluna 2 = 5

Soma Coluna 3 = 6

EXERCÍCIOS

3. Crie um algoritmo que calcule a média dos elementos de uma matriz 5x2.
4. Crie um algoritmo informe qual o maior e qual o menor elemento existente em uma matriz 6x3.
5. Crie um algoritmo que leia uma matriz 3x3 e crie uma segunda matriz que inverta as linhas e colunas da primeira matriz. Ex:

Matriz

1	2	3
4	5	6
7	8	9

Matriz Invertida

1	4	7
2	5	8
3	6	9

EXERCÍCIOS

6. Crie um algoritmo que leia duas matrizes 2x5 e crie uma terceira matriz também 2x5 com o valor da soma dos elementos de mesmo índice. Ex:

Matriz1 + Matriz2 = Matriz3

1	2
3	2
4	1
5	5
1	2

2	4
5	3
7	7
4	4
1	9

3	6
8	5
11	8
9	9
2	11

EXERCÍCIOS

7. Crie um algoritmo que calcule a soma dos valores da diagonal principal de uma matriz 5x5. Veja a **diagonal principal** da matriz destacada no exemplo abaixo:

1	2	5	1	4
3	2	4	2	3
4	1	2	3	7
5	5	2	4	9
1	2	4	5	1

SOMA= 10

EXERCÍCIOS

8. Crie um algoritmo que verifique se uma matriz é **triangular superior**. Uma matriz é triangular superior se todos os elementos abaixo da **diagonal principal** são iguais a 0.

1	2	5	1	4
0	2	4	2	3
0	0	2	3	7
0	0	0	4	9
0	0	0	0	1

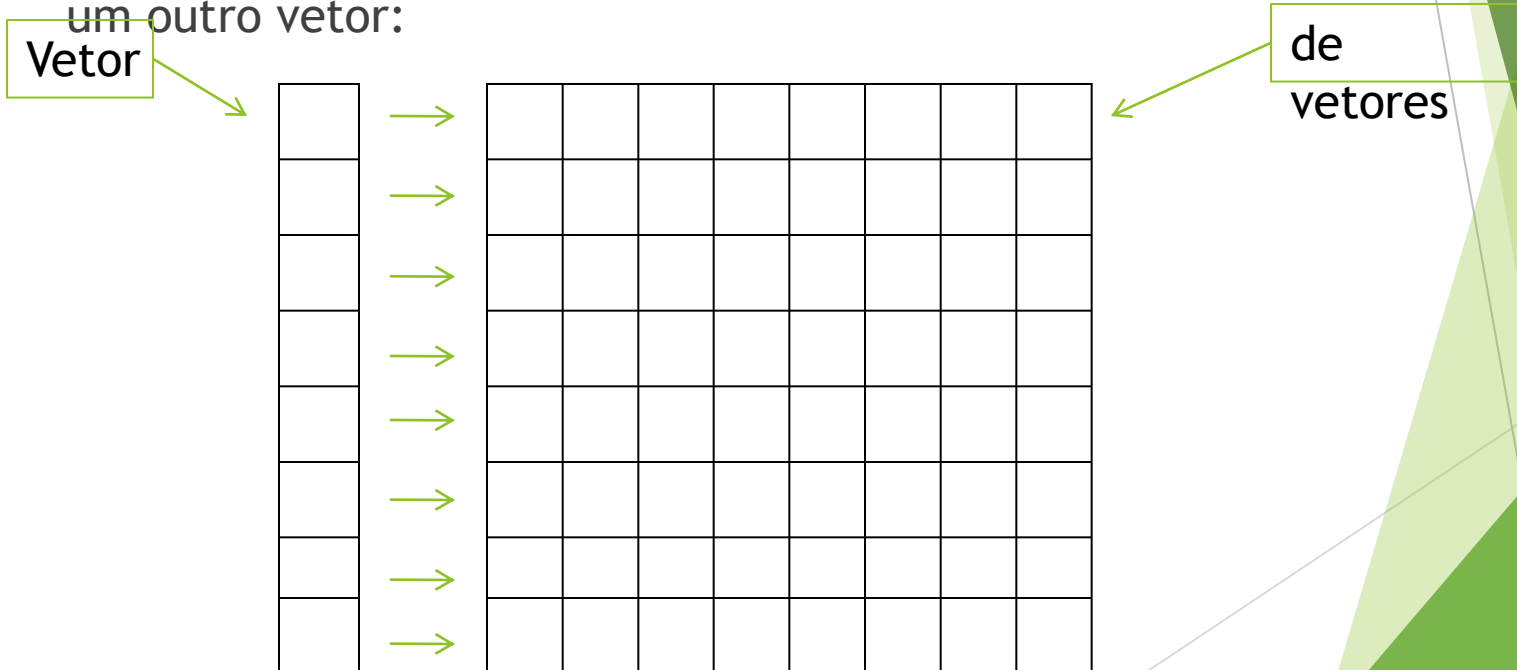
EXERCÍCIOS

9. Crie um algoritmo que verifique se uma matriz é **triangular inferior**. Uma matriz é triangular inferior se todos os elementos abaixo da **diagonal principal** são iguais a 0.

1	0	0	0	0
3	2	0	0	0
4	1	2	0	0
5	5	2	4	0
1	2	4	5	1

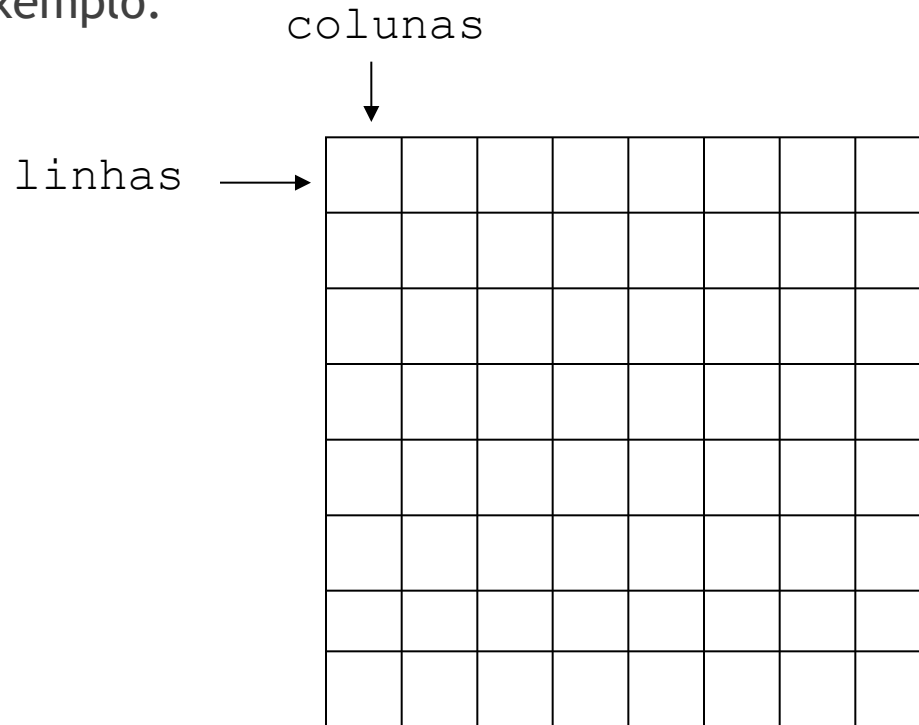
Variáveis Compostas Bidimensionais (matrizes)

- ▶ Mais de um índice para endereçamento.
- ▶ Vetor de vetores: um vetor onde cada posição contém um outro vetor:



Variáveis Compostas Bidimensionais (matrizes)

► Exemplo:



Variáveis Compostas Bidimensionais (matrizes)

► Manipulação:

MATRIZ

	0	1	2	3	4	5	6	7
0								
1								
2								
3								
4								
5								
6								
7								

MATRIZ[2][3]

Variáveis Compostas Bidimensionais (matrizes)

- ▶ Matriz é uma estrutura de dados homogênea bidimensional.
- ▶ Declaração:

tipo nome_da_matriz[m][n];

m: representa o número de linhas da matriz

n: o número de colunas

- ▶ As duas dimensões são, respectivamente, a quantidade de linhas e colunas da matriz
- ▶ Ex: `int mat[10][3];`

Variáveis Compostas Bidimensionais (matrizes)

	col. 1	col. 2	col. 3	...	col. n-1	col. n
linha 1	<code>x[0][0]</code>	<code>x[0][1]</code>	<code>x[0][2]</code>	...	<code>x[0][n-2]</code>	<code>x[0][n-1]</code>
linha 2	<code>x[1][0]</code>	<code>x[1][1]</code>	<code>x[1][2]</code>	...	<code>x[1][n-2]</code>	<code>x[1][n-1]</code>
	...	...	...		...	...
linha m	<code>x[m-1][0]</code>	<code>x[m-1][1]</code>	<code>x[m-1][2]</code>	...	<code>x[m-1][n-2]</code>	<code>x[m-1][n-1]</code>

x é uma matriz bidimensional m x n.

Matrizes: Inicialização



Po

```
float mat[4][3]={ {5.0,10.0,15.0},{20.0,25.0,30.0},  
                  {35.0,40.0,45.0},{50.0,55.0,60.0}};  
  
float mat[][3]= {5.0, 10.0, 15.0, 20.0, 25.0, 30.0,  
                 35.0,40.0,45.0,50.0,55.0,60.0};
```

- ▶ No segundo caso, deve ser informada ao menos a segunda dimensão
- ▶ Usando laços:

```
int mat[3][4];  
int i,j;  
  
//Inicializa a matriz com 1's  
for (i=0; i<3; i++) {  
    for (j=0; j<4; j++) {  
        mat[i][j] = 1;  
    }  
}
```


Matrizes - Impressão

- Exemplo: imprimindo o conteúdo de uma matriz

```
#include <stdio.h>

int main()
{
    int i, j, matriz[3][3] = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    for (i = 0; i < 3; i++)
    {
        for(j = 0; j < 3; j++)
        {
            printf("%d ", matriz[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

```
C:\Z:\UFPE\Graduacao\Disciplinas\ComputacaoEletronica\Exemplos\Mat
1 2 3
4 5 6
7 8 9
Process returned 0 (0x0) execution time : 0.340 s
Press any key to continue.
```

Matrizes passadas por parâmetro

- É necessário especificar a quantidade de colunas da matriz ou todas as dimensões:

```
float media(float a[][2], int lin)
{
    int i;
    float avg, sum=0.0;
    for(i=0; i<lin; ++i)
    {
        for(j=0; j<2; ++j)
            sum+=a[i][j];
    }
    avg = (sum/(lin*2));
    return avg;
}
```

Atividade - Matrizes

1. Construa um algoritmo que efetue e apresente o resultado da soma entre duas matrizes 3 x 5. Inicialize a matriz com valores quaisquer e imprima o resultado na tela.
2. Faça um programa que multiplica uma matriz 3 x 3 de inteiros por um escalar k e imprima o resultado na tela. O usuário deve fornecer os valores da matriz e de k .
3. Leia uma matriz 20 x 20. Leia também um valor X . O programa deverá fazer uma busca desse valor na matriz e, ao final escrever a localização (linha e coluna) ou uma mensagem de “não encontrado”.
4. Dada uma matriz 5x5, elabore um algoritmo que imprima:
 - ▶ A diagonal principal
 - ▶ A diagonal secundária
 - ▶ A soma da linha 4
 - ▶ A soma da coluna 2
 - ▶ Tudo, exceto a diagonal principal
5. Refaça as questões anteriores criando uma função para cada uma delas.

Atividades adicionais - Matrizes

1. Faça um programa para multiplicar duas matrizes com tamanho até 10x10, armazenando o resultado em uma terceira matriz.
 - ▶ O programa deve solicitar ao usuário as duas dimensões das duas matrizes;
 - ▶ O programa deve verificar se as matrizes podem ser multiplicadas e apresentar uma mensagem de erro, caso não seja possível.
2. Refaça o programa anterior transformando apenas o código que faz a multiplicação das matrizes em uma função.
 - ▶ A função recebe como parâmetro as três matrizes e as dimensões das duas primeiras matrizes. O resultado da multiplicação das duas primeiras matrizes deve ser armazenado na terceira matriz.;
 - ▶ A função deve retornar falso se não for possível multiplicar as matrizes, e verdadeiro caso contrário.
 - ▶ A função não deve ler as matrizes, imprimir o resultado, nem a mensagem de erro. Isto deve ser feito na função principal.